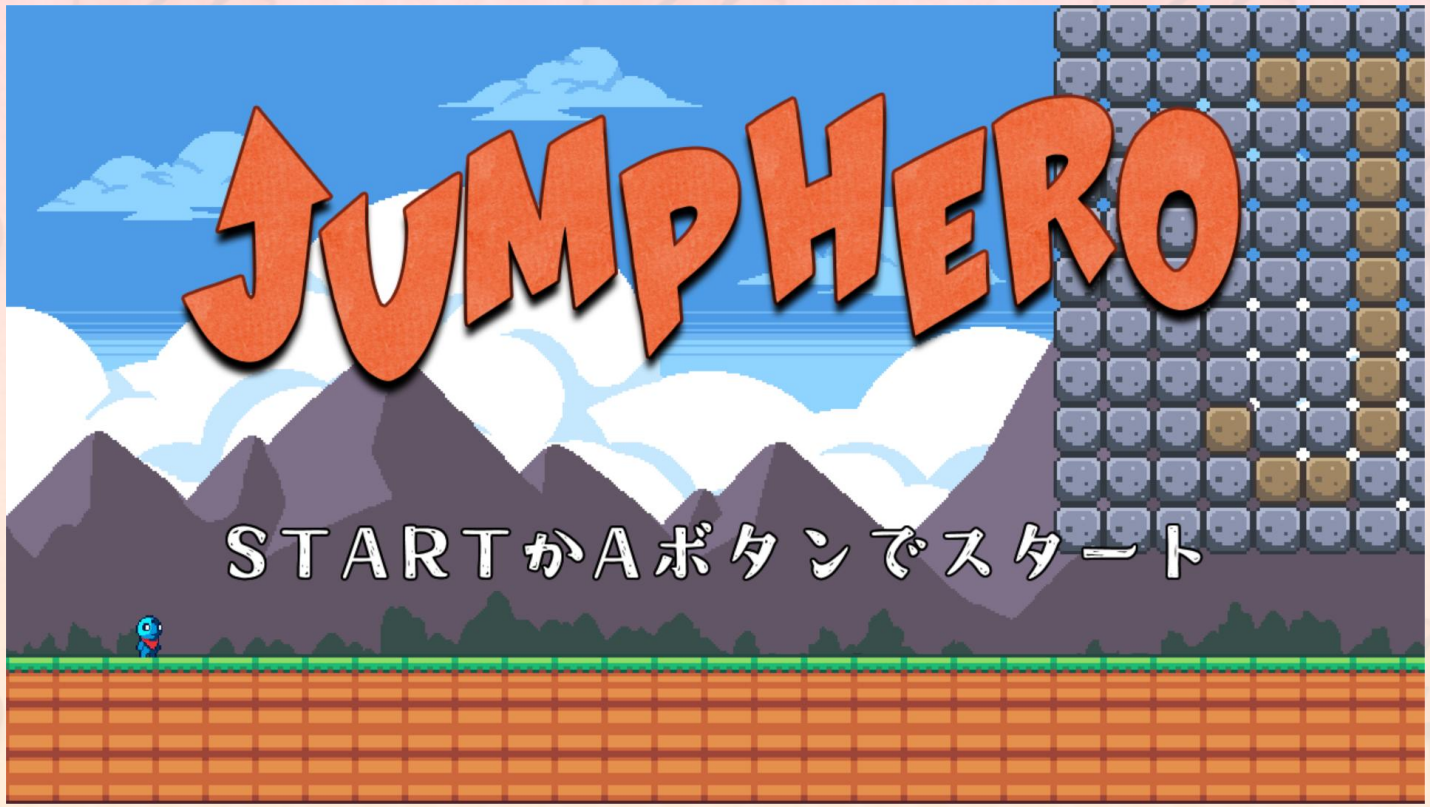
作品紹介

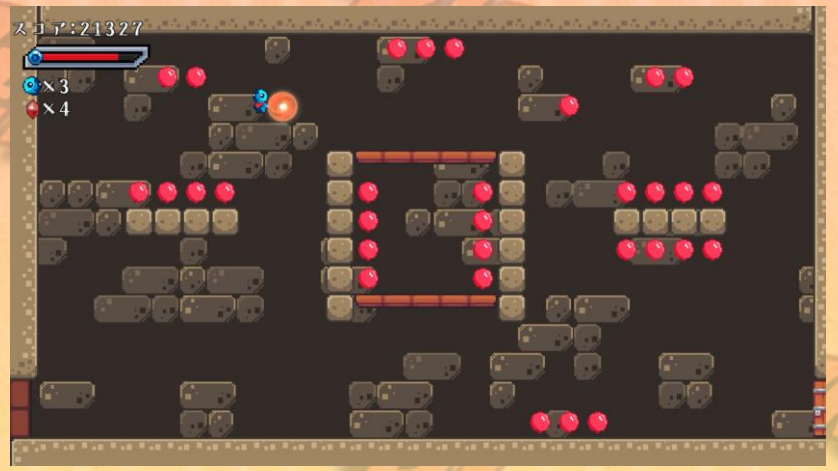


作品名　：JumpHero
ジャンル：2Dアクション
開発環境：DXライブラリ、
Visual Studio 2022
言語　　：C++
制作期間：3か月
制作人数：1人
担当箇所：プログラム全般
動作環境：Windows(XInputコントローラー対応)
GitHub　：https://github.com/KaminagaAyumu/JumpHero.git
動画　　：https://youtu.be/Fodou9HTPLk

ゲーム概要

・ジャンプが得意なヒーローのアクションゲーム
・マップを進んでいき、マップ端の扉に触れると1画面マップに遷移
・1画面マップで風船をすべて取得し、扉に触れるとステージクリア！









こだわった点



こだわった点

～エフェクト～

ジャンプやアイテムを取得した際などに3Dエフェクトを入れ、より派手な演出になるようにしました。



↑画面全体にエフェクトを表示することで派手な演出になっている





こだわった点

～カメラ追従～

画面スクロールに線形補間を使用し、プレイヤーが動いているような表現ができるようにこだわりました。

実際のコード

```
void Camera::Update()
{

    if (m_mode == Mode::None)
    {
        // モードなしの場合ポジション補正を行わない
    }
    else
    {
        const Position2 target = GetCurrentTarget(); // 現在のターゲットを取得
        // カメラの座標を補正
        if (IsLerpEnd()) // 補正が終わっていたら
        {
            m_pos = target; // カメラの座標をターゲットに固定
        }
        else
        {
            m_pos = Geometry::LerpVec2(m_pos, target, kLerpSpeed); // 補正してカメラを動かす
        }
    }
```

少し遅れてプレイヤーに追従する





技術紹介



技術紹介

～外部ファイル化～

アニメーションデータやゲーム中に起こるイベントのデータなど、頻繁に変更するようなパラメータをcsvファイルにまとめ、読み込むことでデバッグの高速化ができるようになりました。

	A	B	C	D	E	F	G	H
1	name	width	height	animNum	animFrame	animIndex	isLoop	
2	Idle	32	32	4	5	0	1	
3	Jump	32	32	6	5	1	0	
4	Walk	32	32	6	5	2	1	
5	Fall	32	32	2	6	3	1	
6								

↑アニメーションのパラメータ

	A	B	C	D	E
1	ID	TriggerType	TriggerParam	ActionType	ActionParam
2	1	GameStart	0	FreezeGame	0
3	2	GameStart	0	ShowText	tuto_start
4	3	GameStart	0	WaitInput	0
5	4	GameStart	0	UnFreezeGame	0
6	5	EnterArea	1	FreezeGame	0
7	6	NoTrigger	0	ShowText	tuto_jump_1
8	7	NoTrigger	0	WaitInput	0
9	8	NoTrigger	0	UnFreezeGame	0
10	9	EnterArea	2	FreezeGame	0
11	10	NoTrigger	0	ShowText	tuto_jump_2
12	11	NoTrigger	0	WaitInput	0
13	12	NoTrigger	0	UnFreezeGame	0
14	13	EnterArea	3	FreezeGame	0
15	14	NoTrigger	3	ShowText	tuto_chest_1
16	15	NoTrigger	3	WaitInput	0
17	16	NoTrigger	0	UnFreezeGame	0
18	17	OpenChest	1	DropItem	coin
19	18	NoTrigger	1	FreezeGame	0
20	19	NoTrigger	1	ShowText	tuto_chest_2

↑イベントのパラメータ（詳細は後述）

技術紹介

～イベント管理1～

ゲーム中で起こるイベントに関してイベント駆動型アーキテクチャを実装し、
チュートリアルやアイテムドロップ処理などを効率よく実装できるようにしました。

```cpp
/// <summary>
/// イベントを起こす条件をまとめた構造体
/// </summary>
struct EventSensors
{
    // プレイヤーが指定されたエリアに入ったかどうかの関数
    std::function<bool(int areaNo)> isEnterAreaFunc;

    // 指定された宝箱を開けたかどうかの関数
    std::function<bool(int chestNo)> isOpenChestFunc;

    // 指定されたアイテムを取得したかどうかの関数
    std::function<bool(const std::string& id)> isGetItemFunc;

    // プレイヤーが強化されているかどうかの関数
    std::function<bool()> isPowerUpFunc;

    // ボタンが押されたかどうかの関数
    std::function<bool()> isPressedButtonFunc;

    // カメラの補間が終わったかどうかの関数
    std::function<bool()> isCameraLerpEndFunc;

    // 風船の取得状態の関数
    std::function<bool(int balloonNum, bool isAll)> isGetBalloonFunc;

    // 敵のスポーン位置すべてを取得する関数
    std::function<const std::vector<Position2>&()> getSpawnPositionsFunc;

    // 風船の総数を取得する関数
    std::function<int()> getTotalBalloonNumFunc;
};
```

↑イベントが起こる条件フラグ用の関数オブジェクト

```cpp
/// <summary>
/// イベントの内容をまとめた構造体
/// </summary>
struct EventControls
{
    // ヒントを表示する関数
    std::function<std::weak_ptr<UITextWindow>(const std::string& id, const Size& size, co

    // テキストウィンドウを表示する関数
    std::function<std::weak_ptr<UITextWindow>(const std::string& id, const Size& size, co

    // アイテムをドロップする関数
    std::function<void(int chestNo, const std::string& itemType)> dropItemFunc;

    // 敵を生成する関数
    std::function<void(const std::vector<Position2>& pos, int formNo)> spawnEnemiesFunc;

    // プレイヤーのフリーズ状態を変更する関数
    std::function<void()> changePlayerFreezeFunc;

    // カメラが見るターゲットをセットする関数
    std::function<void(const std::string& key)> lookCameraFunc;
```

↑フラグが立った際に起きるイベント用の関数オブジェクト

技術紹介

～イベント管理2～

ゲーム中で起こるイベントに関してイベント駆動型アーキテクチャを実装し、
チュートリアルやアイテムドロップ処理などを効率よく実装できるようにしました。

```cpp
/// <summary>
/// ゲーム中のイベントを管理するクラス
/// </summary>
class EventManager
{
public:
    EventManager();
    virtual ~EventManager();

    void Update();
    void Draw() const;

    /// <summary>
    /// イベントをセットする
    /// </summary>
    /// <param name="controls"></param>
    /// <param name="sensors"></param>
    void SetEvents(const std::shared_ptr<EventControls>& controls, const std::shared_ptr<EventSensors>& sensors);

    /// <summary>
    /// 指定されたステージのイベントデータをロードする
    /// </summary>
    /// <param name="stageNo">ステージ番号</param>
    /// <param name="isMiniGame">ミニゲームかどうか</param>
    bool LoadEventData(int stageNo, bool isMiniGame);

    /// <summary>
    /// 指定されたステージの共通イベントデータをロードする
    /// </summary>
    /// <param name="stageNo">ステージ番号</param>
    /// <param name="isMiniGame">ミニゲームかどうか</param>
    bool LoadCommonEventData(int stageNo, bool isMiniGame);

private:

    std::vector<EventData> m_eventData; // イベントデータ群
    std::vector<CommonEventData> m_commonEventData; // 共通イベントデータ群
```

↑イベントをvectorで管理するクラス(関数オブジェクトを登録できる)

```cpp
bool EventManager::CheckTrigger(const EventData& data)
{
    // 関数を呼ぶためにlockする
    auto sensors = m_pSensors.lock();
    if (!sensors) return false; // lock出来なかった場合処理を終わる

    // イベントが始まる条件によってフラグを分岐
    switch (data.triggerType)
    {
    case TriggerType::GameStart:
        return true;
        break;
    case TriggerType::EnterArea:
    {
        int areaNo = GetParamNum(data.triggerParam);
        return sensors->isEnterAreaFunc(areaNo);
    }
    break;
    case TriggerType::TimeElapsed:
    {
        m_timeCount++;
        int endTime = GetParamNum(data.triggerParam);
        if (m_timeCount >= endTime)
        {
            m_timeCount = 0;
            return true;
        }
    }
    break;
    case TriggerType::EventEnd:
    {
        int eventNo = GetParamNum(data.triggerParam);
        return IsEventEnd(eventNo);
    }
    case TriggerType::OpenChest:
    {
        int chestNo = GetParamNum(data.triggerParam);
        return sensors->isOpenChestFunc(chestNo);
    }
```

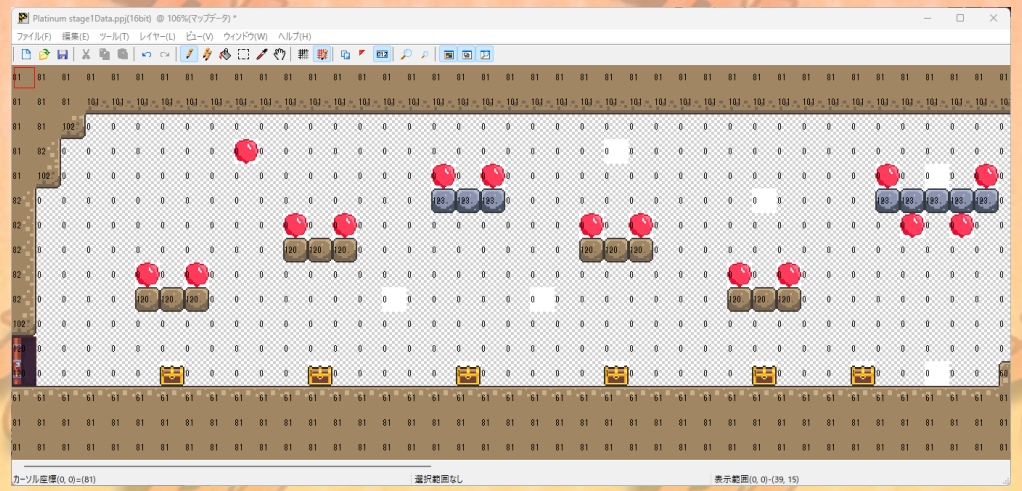
↑関数オブジェクトでフラグを管理する

技術紹介

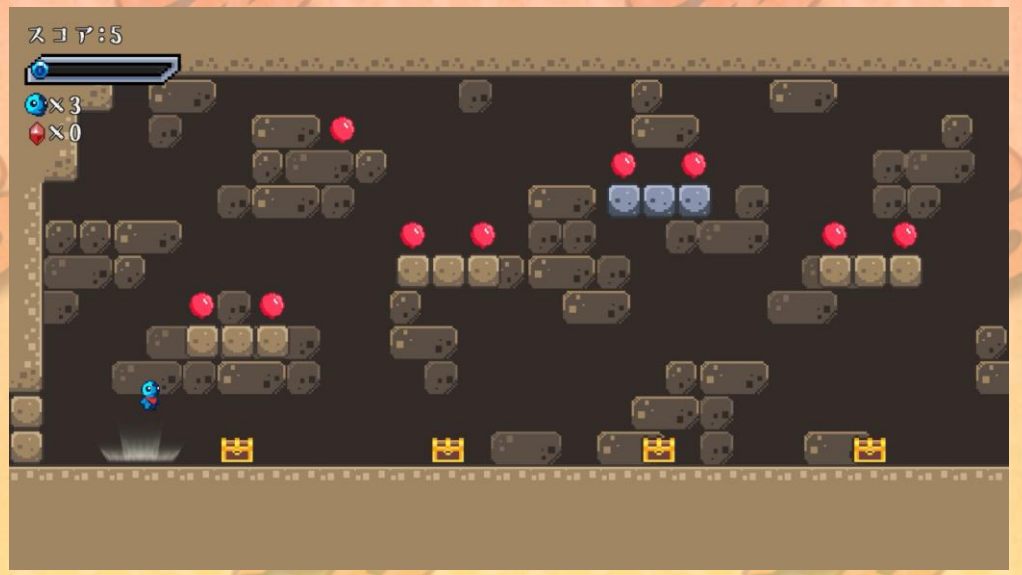
～バイナリデータ1～

マップのデータをバイナリデータ化し、地形データ、アイテムの配置、敵のスポーン位置を分けることで拡張性のあるマップを実装しました。

↓Platinum(外部ツール)



バイナリ
読み込み



技術紹介

～バイナリデータ2～

ハイスコアデータをバイナリデータとして保存、読み込む処理を実装することで、ゲームを閉じてもデータが記録されるようにしました。

```
void ScoreManager::Save()
{
    FILE* fp = nullptr;

    // バイナリデータを保存する
    auto err = fopen_s(&fp, kScoreFileDataName, "wb");

    if (fp == nullptr)
    {
        return;
    }

    ScoreHeader header = {};

    header.signature[0] = 's';
    header.signature[1] = 'c';
    header.signature[2] = 'd';
    header.signature[3] = 't';

    header.version = 1.0f;
    header.dataNum = sizeof(m_data);

    // ヘッダへの書き込み
    fwrite(&header, sizeof(header), 1, fp);

    // 実際のデータへ書き込み
    fwrite(&m_data, sizeof(m_data), 1, fp);

    // ファイルを閉じる
    fclose(fp);
}
```

技術紹介

～オブジェクト指向～

基底クラスから継承させていくポリモーフィズムを用いたクラス設計を行い、
オブジェクト指向を意識した設計を行いました。

